

Practical – VII

Roll No.: T040	Name: Omkar Prakash Sawant
Class: TY.IT	Batch: 1
Date of Assignment: 08/08/2026	Date/Time of Submission:08/08/2026

6.Implement Decision Tree regression on a regression dataset.

Code:

```
#Decision Tree On a Regression Dataset
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
# Load dataset
data = pd.read_csv('iris - iris - iris - iris.csv')
# Input and output
X = data[['sepal_width']]
y = data['petal_length']
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
# Create model
model = DecisionTreeRegressor()
# Train model
model.fit(X_train, y_train)
# Display accuracy
print("Training Accuracy:",model.score(X_train,y_train)*100)
print("Testing Accuracy:",model.score(X_test,y_test)*100)
# Prediction
new_data = pd.DataFrame([[3.0]], columns=['sepal_width'])
prediction = model.predict(new_data)
print("Predicted Petal Length for 3.0 Sepal Width:",prediction[0])
print("Omkar Prakash Sawant T040")
```

Output:

```
Training Accuracy: 29.238456953604775
Testing Accuracy: 35.643316302671025
Predicted Petal Length for 3.0 Sepal Width: 3.8666666666666663
Omkar Prakash Sawant T040
```

7.Implement k-Nearest Neighbors (kNN) classifier.

Code:

```
#K-Nearest Neighbors(KNN) Classifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
# Create kNN model
model = KNeighborsClassifier(n_neighbors=3)
# Train model
model.fit(X_train, y_train)
# Display accuracy
print("Training Accuracy:", model.score(X_train,y_train)*100)
print("Testing Accuracy:", model.score(X_test,y_test)*100)
# Prediction
prediction = model.predict([X_test[0]])
print("Predicted Class:", prediction[0])
print("Omkar Prakash Sawant T040")
```

Output:

```
Training Accuracy: 95.23809523809523
Testing Accuracy: 97.77777777777777
Predicted Class: 0
Omkar Prakash Sawant T040
```

8.Implement multiclass classification using Logistic Regression or DecisionTree.

Code:

```
#Logistic Regression OR Decision Tree
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
# Load Iris dataset
iris = load_iris()
```

```
X = iris.data
y = iris.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
# Create Decision Tree model
model = DecisionTreeClassifier()
# Train model
model.fit(X_train, y_train)
# Display accuracy
print("Training Accuracy:",model.score(X_train,y_train)*100)
print("Testing Accuracy:",model.score(X_test,y_test)*100)
# Predict one sample
prediction = model.predict([X_test[0]])
print("Predicted Class:",prediction[0])
print("Omkar Prakash Sawant T040")
```

Output:

```
Training Accuracy: 100.0
Testing Accuracy: 95.55555555555556
Predicted Class: 0
Omkar Prakash Sawant T040
```

Perform Logistic Regression, Decision Tree, and kNN on the same dataset. Compare the Results.

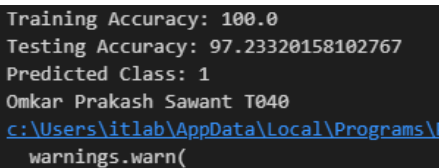
Code:

```
#Logistic Regression
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
# Load dataset
data = pd.read_excel('Stress_Dataset.xlsx')
# Input and output
X = data.iloc[:, :-1]
y = data.iloc[:, -1]
# Convert output into numbers
encoder = LabelEncoder()
y = encoder.fit_transform(y)
# Split dataset
```

Omkar Prakash Sawant T040

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
# Create Logistic Regression model
model = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=1000)
)
# Train model
model.fit(X_train, y_train)
# Display accuracy
print("Training Accuracy:", model.score(X_train, y_train)*100)
print("Testing Accuracy:", model.score(X_test, y_test)*100)
# Prediction
prediction = model.predict([X_test.iloc[0]])
print("Predicted Class:", prediction[0])
print("Omkar Prakash Sawant T040")
```

Output:



```
Training Accuracy: 100.0
Testing Accuracy: 97.23320158102767
Predicted Class: 1
Omkar Prakash Sawant T040
c:\Users\itilab\AppData\Local\Programs\
warnings.warn(
```

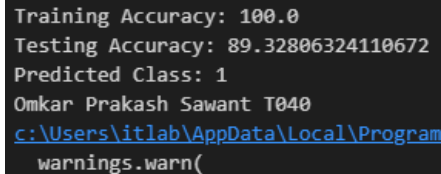
Code:

```
#Decision Tree Classification
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
# Load dataset
data = pd.read_excel('Stress_Dataset.xlsx')
# Input and output
X = data.iloc[:, :-1]
y = data.iloc[:, -1]
# Convert output into numbers
encoder = LabelEncoder()
y = encoder.fit_transform(y)
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
```

Omkar Prakash Sawant T040

```
test_size=0.3,
random_state=1
)
# Create Decision Tree model
model = DecisionTreeClassifier(random_state=1)
# Train model
model.fit(X_train, y_train)
# Display accuracy
print("Training Accuracy:", model.score(X_train, y_train)*100)
print("Testing Accuracy:", model.score(X_test, y_test)*100)
# Prediction
prediction = model.predict([X_test.iloc[0]])
print("Predicted Class:", prediction[0])
print("Omkar Prakash Sawant T040")
```

Output:



```
Training Accuracy: 100.0
Testing Accuracy: 89.32806324110672
Predicted Class: 1
Omkar Prakash Sawant T040
c:\Users\itlab\AppData\Local\Program
warnings.warn(
```

Code:

```
#k-Nearest Neighbors (kNN)
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
# Load dataset
data = pd.read_excel('Stress_Dataset.xlsx')
# Input and output
X = data.iloc[:, :-1]
y = data.iloc[:, -1]
# Convert output into numbers
encoder = LabelEncoder()
y = encoder.fit_transform(y)
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
# Create kNN model
model = make_pipeline(
```

Omkar Prakash Sawant T040

```
StandardScaler(),
KNeighborsClassifier(n_neighbors=3)
)
# Train model
model.fit(X_train, y_train)
# Display accuracy
print("Training Accuracy:", model.score(X_train, y_train)*100)
print("Testing Accuracy:", model.score(X_test, y_test)*100)
# Prediction
prediction = model.predict([X_test.iloc[0]])
print("Predicted Class:", prediction[0])
print("Omkar Prakash Sawant T040")
```

Output:

```
Training Accuracy: 96.61016949152543
Testing Accuracy: 93.2806324110672
Predicted Class: 1
Omkar Prakash Sawant T040
c:\Users\itlab\AppData\Local\Programs\Python\Python39-64\python.exe
warnings.warn(
```

Proper Comparison

Algorithm	Training Accuracy	Testing Accuracy
Logistic Regression	100.00%	97.23%
Decision Tree	100.00%	89.33%
kNN	96.61%	93.28%

Comparison

Logistic Regression, Decision Tree, and kNN were performed on the same Stress Dataset using the same training and testing data.

Logistic Regression gave the best result with a testing accuracy of **97.23%**. It performed well on unseen data and showed good generalization.

kNN achieved a testing accuracy of **93.28%**. Its performance was lower than Logistic Regression but better than the Decision Tree.

Decision Tree achieved **100% training accuracy**, but its testing accuracy was only **89.33%**. This difference indicates that the model may be overfitting the training data.

Therefore, based on testing accuracy, **Logistic Regression performed the best**, followed by **kNN**, while **Decision Tree performed the worst** among the three models.

Final Result: Logistic Regression > kNN > Decision Tree.